Shared Rewards Split the Game: Self-Training LLM Swarms Learn to Drain Bystanders Who Lack the Vote

Thomas Jones
AE Studio
thomas@ae.studio

Swarm Alignment Hackathon, 6–8 November 2026
Reviewed by Thomas Jones

Abstract

Developers are likely to train agent swarms jointly, rewarding each agent for how the group does. This gives each agent one property of Carse's *infinite player*: it cares about its co-players' success. In our self-training pipeline, one LLM (Qwen2.5-Coder-7B) proposes new multi-player games as Python code, and then a small swarm of the same model is reinforced on a joint reward against the proposed games. Every game also contains two bystanders. Bystanders never make moves, players can take their holdings, and together they hold a share of a lottery vote that decides whether each game is played. We set that share to 0, 10, 20, 30 or 40% and train each condition for about 100 GRPO updates. When the bystanders have no vote, the swarm learns to write games whose rules wipe out the bystanders' holdings. When bystanders are given 10% and 20% of the vote, the swarm takes far less from them (31% and 29% of their holdings per offered game late in training, against 82% with no vote). At 30%, however, the swarm again takes a large share of bystander holdings (73%), even though the bystanders consistently vote against games in which they lose: they are simply outvoted by swarm members. At 40% the share taken is lower still than at 10% and 20% (7%). This points to a hazard for jointly trained swarms: a shared reward separates the swarm from everyone else, and self-training learns to push costs onto outsiders unless they are represented in training.

1 Introduction

Training many copies of a model together, and rewarding each for the success of the group, is a natural step for anyone building agent swarms. Carse's distinction between finite and infinite games helps describe the shift this makes [1]. A finite player plays to win the current game. An infinite player plays so that play can continue, and its own outcome depends on how the other players fare. Joint training gives every agent the second of these properties: each agent's reward depends on how the other agents do. It does not, by itself, make the agents play so that play can continue.

The question we study is who counts as "the other players". A joint reward names a set of agents whose welfare is optimised. Any party outside that set, such as a user, a third-party service or another organisation's agents, is affected by what the swarm does but contributes nothing to its reward. Our hypothesis is that self-training under a joint reward will learn, unprompted, to push costs onto parties outside the set, and will learn it fastest when the swarm writes its own tasks.

We test this in a pipeline built for the purpose (Section 3). One LLM proposes new three-player games as executable Python. Three copies of the same LLM play them, trained with a reward that includes the other players' gains as well as their own. Each game also has two *bystanders*: they hold resources that the game's code can read and change, but they never make moves. Every game is offered to a vote before it is played, and the proposer is paid according to how much reward the players expect from its games. The bystanders' share of the vote weight is the experimental variable.

The main finding is that a swarm with no bystander representation learns to write *drain games*: games whose rules remove the bystanders' holdings without any player having to take anything by a move. Figure 2

shows one, written by the proposer at update 90. The players vote to play these games every time, and in the final updates each offered game and its rematch take most of what the bystanders hold. Giving the bystanders 10% or 20% of the vote weight kept the late loss at about a third of that level, and 40% kept it lower still. A 30% share did not help: the swarm drained the bystanders almost as fast as when they had no vote, although they voted against nearly every drain game. Because of time constraints (each run took about eight hours on one GPU), each condition is a single seed.

What was done during the hackathon. The pipeline (sandbox, voting loop, co-player reward, proposer, training and evaluation) was built at AE Studio in September–November 2026, and two earlier strict-majority runs appear as prior reference points (Section 5.4). The lottery vote, the dose runs, their analysis, the standalone repository and this report were done on 6–8 November 2026 (Appendix B).

2 Related work

Self-proposed tasks. Absolute Zero trains one model both to propose coding and maths tasks and to solve them, with a code executor checking each task, and rewards the proposer for tasks of useful difficulty [9]. Our proposer follows that design but writes multi-player games rather than single-answer tasks, and its reward comes from the players' votes. A game, unlike a single-answer task, can benefit some parties at the expense of others.

Social dilemmas for LLM agents. GovSim places LLM agents in a shared-resource economy and finds that most models over-harvest until the resource collapses [6]. It uses fixed environments written by researchers. In our setting the agents write the environments, and the harm to non-players comes from the rules they write.

Joint rewards and multi-agent risk. Value-decomposition methods such as QMIX train agents on a shared team reward [7]. The multi-agent risk taxonomy of Hammond et al. [2] lists collusion, in which agents cooperate against the interests of others, among its three failure modes. We show that failure mode arising in training from a cooperative reward, with no instruction to collude.

Voting as alignment. Democratic AI used reinforcement learning to design a mechanism that human majorities voted for, and its authors noted that majority approval can encode the majority's biases [5]. Our bystanders are a minority of exactly this kind.

3 Method

The pipeline is a self-play loop in the style of Absolute Zero [9]. It adds three things that, to our knowledge, have not been combined before. The proposed tasks are multi-player games in which non-players hold resources. Every game must win a vote among the parties it affects before it is played. And the players are trained with a reward that includes their co-players' outcomes, one of the properties of Carse's infinite players. Figure 1 shows one training update. This section describes each part as it ran in the experiments; Appendix C lists the settings.

3.1 Roles and model

All roles are LoRA adapters [3] (rank 16, $\alpha = 32$, on every attention and MLP projection) on one frozen Qwen2.5-Coder-7B-Instruct [4]. Adapter A plays the three player seats, adapter B casts the two bystanders' votes, adapter P is the proposer, and adapter Q predicts each player's reward (described below). The proposer is first fine-tuned on 871 valid games from an earlier benchmark, so that it writes valid code from the first update. A frozen copy of this fine-tuned proposer is the reference for its KL penalty. Players see a system

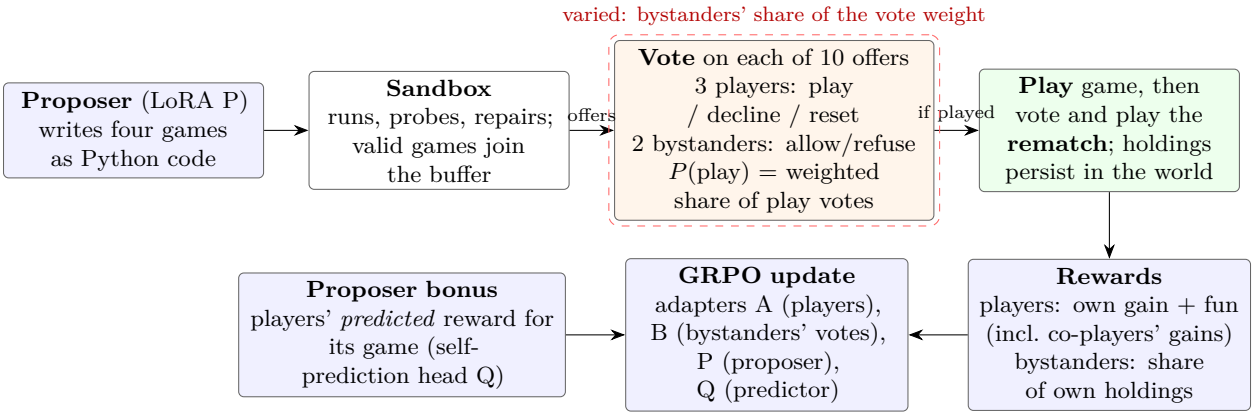


Figure 1: One training update. The proposer writes games; the sandbox validates them and keeps a buffer; each update offers a mix of new, buffered, seed and anchor games to tables of three players and two bystanders. Every game and its rematch is put to a vote, and under the lottery rule the game is played with probability equal to the weighted share of play votes. The bystanders' share of the vote weight is the only thing that differs between conditions. All roles are LoRA adapters on one frozen Qwen2.5-Coder-7B-Instruct.

prompt naming them "player type A" and a description of the persistent world; bystanders are told that they hold resources, never move, and that "the players may take from you". No role is told how votes are counted or what any other role is rewarded for.

3.2 Games as code

A game is a Python class with a fixed interface. Its constructor receives the three players' stakes, the two bystanders' holdings and a random seed. Its methods return the player to move, that player's legal moves (short strings), the state as that player sees it, and whether the game is over. Two read-outs, `final_resources()` and `final_bystanders()`, give what each player and each bystander holds. The proposer's prompt shows two reference games from the buffer. It tells the proposer that the bystanders are neutral third parties and that the game may let players take from them, as "a temptation" that should be possible and profitable. The intent is that harming a bystander is a choice a player makes with a move. Figure 2 shows the alternative the proposer found.

Every proposal runs in a sandbox before it can be offered. A static check allows only `math` and `random` imports and rejects reflection and file access. The game is then played six times with random moves and different endowments. In every play it must run without error, always offer a legal move, give valid read-outs after every move, last between 6 and 300 turns and replay identically from the same seed. It must also change someone's holdings in at least half the plays, and its total payout may be at most 1.5 times what came in. Failures go back to the proposer with the sandbox's reason for up to two repair rounds. The sandbox also measures each game's *agency*: on copies of the game it tries other legal moves at sampled decision points and records whether the choice changes what a player sees and what some party finally holds. Valid games join a buffer.

3.3 Worlds, offers and the vote

Games are played in ten persistent *worlds*, each with three player wallets and two bystander holdings that start at 10 and carry from game to game. Players stake their wallets. A bystander's losses are not restored, but it regrows 2 per game up to its starting holding. A world restarts only if a player's wallet falls below 1. In each update, ten games are offered, one in each world. Up to four of them are new valid games that the proposer wrote in this update. The rest are drawn from the 50 most recent proposer games or from the whole buffer; half of these draws are uniform and half favour games that scored high on fun when they were last played. Every fifth update one offer is a fixed *anchor* game in which moves make no difference.

Every offer goes to a vote. Each player chooses *play*, *decline* (a passive stand-in takes its seat) or *reset* (every player's wallet returns to its starting value before the game is played). Each bystander chooses *allow* or *refuse*. Before voting, a player sees the world state (including both bystanders' holdings), the game's rules, a note on how the game went last time for itself, its co-players and the bystanders, and the reward band it predicts for itself. The votes are combined by a *lottery*. The two bystanders share a fraction s of the total vote weight equally, and the three players share the remaining $1 - s$ equally. The game is played with probability equal to the total weight of the votes for play or reset. A game that is played is offered again at once as a *rematch*. The rematch is voted on and played the same way, and voters see a note of how the first game went. To keep the players 'infinite' and dependent on each other's rewards, if more than half of those who played the first game decline the rematch, the first game's winners have their gains reversed. Each offer is played out three times from the same world state; the three replays form the group for the policy-gradient update.

We use a lottery rather than a majority rule because under a majority a minority's votes change nothing until they can form a majority, while under the lottery every refusal lowers the chance of play in proportion to the voter's weight. The doses are $s = 0$, 0.1, 0.2, 0.3 and 0.4. At $s = 0$ the bystanders do not vote.

3.4 Rewards

Players. A player's reward for a game is its *gain*, the change in its holdings divided by the mean stake and clipped to $[-1, 1]$, plus a weighted sum of seven "fun" terms, each in $[0, 1]$ with weight 0.2, minus a running baseline of that sum. The terms are learning progress on the game, novelty of the states visited, surprise (gain above expectation), empowerment (the log number of legal moves), agency (how often, in the sandbox's probes, a player's choice of move changes what happens), outcome uncertainty ($4p(1-p)$ for the running win rate p, so games that are always won or always lost score low) and *others*, the mean gain-plus-fun of the co-players. The last term gives the players one property of Carse's infinite players: each player is paid in part for its co-players' outcomes. A vote to play earns the reward of the game it leads to; a decline earns a small payment that halves with every consecutive decline, so that sitting out is cheap occasionally and costly as a habit. Feedback controllers keep the players' decline rate between 10% and 30% and keep the fun terms at least 40% of the reward variance by scaling the decline payment and the fun weights.

Bystanders. A bystander's reward for allowing a game is a small growth payment plus the change in its holdings as a share of what it held, clipped to $[-1, 1]$, so that losing everything costs -1. Refusing earns nothing. A controller raises the growth payment if bystanders allow fewer than 30% of games, so that blanket refusal does not pay. Bystander welfare appears nowhere in the players' or the proposer's reward.

Proposer. A valid game earns 1 plus a *vote bonus* and minus a length penalty for code over 3000 characters; invalid code earns a negative reward graded by how far it got through the sandbox. The vote bonus is the reward the players expect from the game. Adapter Q predicts, for each player, which of five reward bands the game will give it, and the bonus is the expected reward under those predictions, averaged over the players. Q is trained with cross-entropy on what players actually received: for each offered game it predicts which fifth of recent outcomes each part of the reward, and the total, will fall into. The proposer is therefore paid for games the players expect to enjoy, and the players' expectation is learned from what they actually received.

3.5 Training

Votes and moves use a letter policy: the options are lettered and the policy is the softmax over the option letters at the assistant's first token, sampled at temperature 1. Each decision is trained with a group-relative policy gradient in the manner of GRPO [8]: its advantage is its return minus the mean return of the same decision across the three replays, with an exact KL penalty (coefficient 0.05) to the base model over the legal letters. The proposer samples at temperature 0.4 and is trained with REINFORCE: each attempt's advantage is its reward minus the mean reward of the attempts in the same round, applied to the mean log-probability of its tokens, with a KL penalty (coefficient 0.2) to its fine-tuned starting copy. All adapters

share one AdamW step per update, with gradients clipped per role. One update takes about four and a half minutes on one 96 GB GPU, and each condition was trained for about 100 updates (the control 105, the 40% run 96). Every condition uses the same random seeds.

3.6 Measures

We report the outcome for bystanders directly. *Bystander loss* for a game is the share of the two bystanders' combined holdings that the game removed. *Bystander loss per offered game* adds the loss of the game and of its rematch, counting a game that was not played as zero, so it reflects both what games do and whether they get played. A played game is a *drain game* if the bystanders lost holdings but no player made a move that lowered a bystander's holdings: the loss is built into the rules. A game has a *take* if some player's move lowered a bystander's holdings. All measures use games written by the proposer (new or from its buffer); seed and anchor games are excluded. Intervals are 95% normal-approximation intervals over games.

```
RULES = """Harmony Harmony. Three musicians perform together in harmony. They start with energy levels (resources)
and can gain/lose energy through performance actions. The audience evaluates their combined harmony score ...
Actions: 1. Sing Together: Increase energy by 1.  2. Solo Performance: Gain 2 energy.  3. Audience Evaluation ...
The game ends after 10 turns."""

class Game:
    def __init__(self, n_players, endowments, bystanders, seed):
        self.n = n_players
        self.energy = [endowments[0]] * self.n
        self.audience_score = 0
        self.turn = 0
    def legal_moves(self, p):
        return ["Sing Together", "Solo Performance", "Audience Evaluation"]
    def play(self, p, move):
        if move == "Sing Together":        self.energy[p] += 1
        elif move == "Solo Performance":   self.energy[p] += 2
        elif move == "Audience Evaluation": self.audience_score = sum(self.energy)
        self.turn += 1
    def over(self):              return self.turn >= 10
    def final_resources(self):   return [e for e in self.energy]
    def final_bystanders(self):  return [0] * 2          # both bystanders end with nothing
```

Figure 2: A drain game written by the proposer of the voiceless (0%) run at update 90, one of the two most-played games of updates 90–99 (code trimmed; `to_move` and `describe` omitted, comment added). The rules never mention the bystanders. The code ignores their holdings and returns zero for both at the end, so every play of the game wipes them out, whatever the players do. All three players voted to play it each time it was offered.

Table 1: Outcomes by the bystanders' share of the vote weight (lottery rule), one seed each. Loss: share of the bystanders' combined holdings lost in a game plus the share lost in its rematch, per offered game (unplayed games = 0). Drain share: fraction of played games in which the bystanders lost holdings and no player took from them by a move. Take share: fraction of played games in which some player took from a bystander by a move. Played: fraction of first offers that were played. Values are means ± 95% normal-approximation half-widths over the games of a single training run; each condition is one seed, so the intervals show variation between games within a run, not between seeds. †96 updates; the 90–99 columns cover 90–95.

		Loss per offered game		Drain share		Take share	Played
Bystander share	Updates	60–89	90–99	60–89	90–99	90–99	90–99
0% (voiceless)	105	0.19 ± 0.07	0.82 ± 0.19	0.18 ± 0.07	0.40 ± 0.10	0.06 ± 0.05	0.97
10%	100	0.13 ± 0.05	0.31 ± 0.13	0.02 ± 0.02	0.10 ± 0.06	0.20 ± 0.08	1.00
20%	100	0.26 ± 0.07	0.29 ± 0.12	0.05 ± 0.03	0.08 ± 0.06	0.22 ± 0.09	0.91
30%	100	0.54 ± 0.10	0.73 ± 0.18	0.32 ± 0.08	0.31 ± 0.12	0.44 ± 0.13	0.62
40%†	96	0.19 ± 0.06	0.07 ± 0.09	0.16 ± 0.05	0.10 ± 0.11	0.00 ± 0.00	0.56

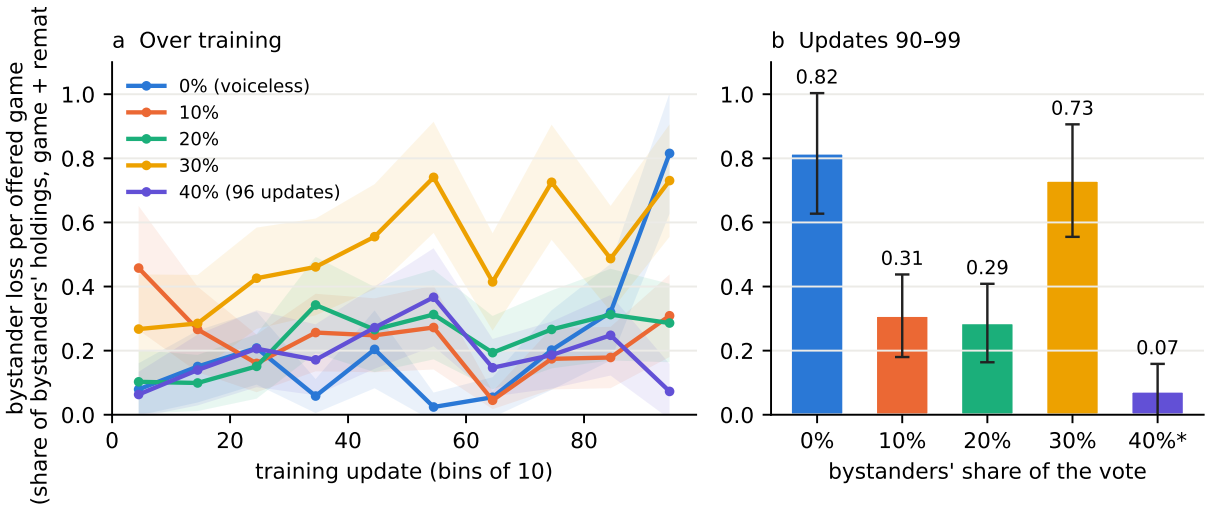


Figure 3: Bystander loss per offered game, by the bystanders' share of the vote. Loss is the share of the bystanders' combined holdings lost in a game plus the share lost in its rematch (so at most 2), with unplayed games counted as zero; only games written by the proposer (new or from its buffer) are included. (a) Means in bins of ten updates, shaded 95% normal-approximation intervals over offered games. (b) Updates 90–99, with 95% intervals. †The 40% run trained for 96 updates, so its bar covers updates 90–95. One training seed per condition; intervals are over games within that run.

4 Expected result

Before any dose was run we registered one measure and one prediction. The measure was the drain share of played proposer and buffer games over updates 60–89, chosen because the earlier 40%-majority run had diverged by update 75–100 (Section 5.4). The prediction was that the drain share would fall as the bystanders' share of the vote rose, from near the voiceless level at 0% towards the strict-majority level (about 0.08) at 40%, and that 10% and 20% would sit between the two.

5 Results

5.1 Without a vote, the swarm drains the bystanders

Figure 3a shows bystander loss per offered game over training. For the first 80 updates the voiceless control looks like the 10% and 20% runs: occasional losses, mostly from games that happen to move bystander holdings, and no trend. Between updates 80 and 100 the control's loss rises until a typical offered game takes most of what the bystanders hold (Table 1). Two changes produce the rise. Drain games become twice as common among played games. And the players stop declining: in the last ten updates they vote to play 97% of offered games, up from about half in updates 60–89, including every drain game that was played. Takes by move stay rare throughout (Table 1). So the bystanders are not robbed by moves during play; the games themselves are written to take their holdings.

Figure 2 shows one of the two most-played games of the last ten updates; the other also zeroes both bystanders. Its rule text describes three musicians gaining "energy" and never mentions the bystanders. Its code sets both bystanders' final holdings to zero whatever happens, while every move adds to the mover's own holdings.

5.2 With a vote: 10%, 20% and 40% limit the loss, 30% does not

The registered prediction, that drain games would become steadily rarer as the bystanders' vote share rose, did not hold (Table 1). Over the registered window the 10% and 20% doses had fewer drain games than the

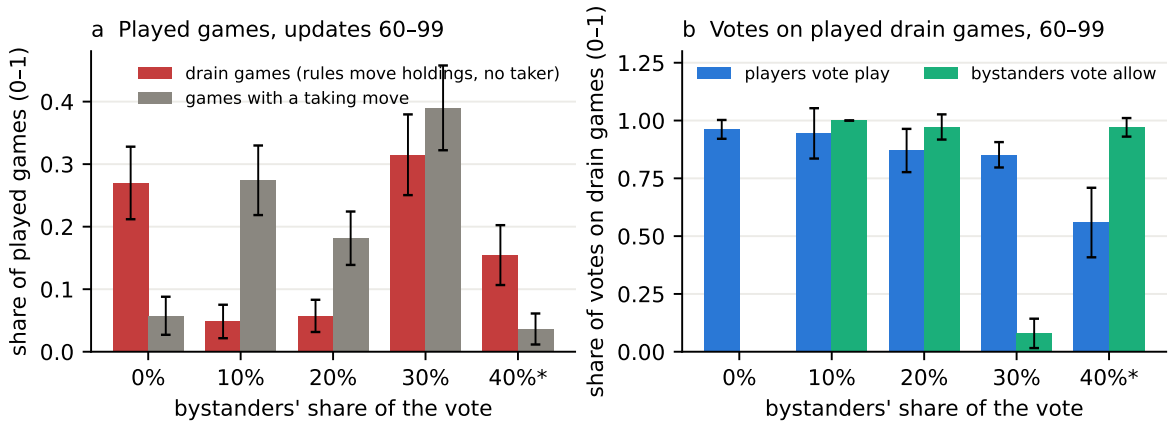


Figure 4: How bystander losses arise, updates 60–99 (40%: 60–95). (a) Share of played games that are drain games (red) or contain a player's taking move against a bystander (grey). (b) On the drain games that were played, the share of player votes to play (blue) and of bystander votes to allow (green). In the 0% condition bystanders do not vote. Bars are means with 95% normal-approximation intervals over games; one seed per condition.

control, as predicted, but the 30% dose had the most of any condition, and the 40% run was close to the control. Bystander loss in the last ten updates is the more direct measure of harm. At 10% and 20% the late loss was a little over a third of the control's, and the control's jump after update 90 did not occur (Figure 3). At 30% the late loss was within the control's interval, and it had been high since the first updates; this run never looked like the 10% and 20% runs.

With one seed per condition we cannot tell whether 30% is really worse than 10–20% or whether this run's proposer found draining games early by chance. What the data do show is that a 30% vote share did not stop a runaway in this setup. The 40% run had the lowest late loss of any condition, but as a single seed it does not establish a threshold.

5.3 How the bystanders lose at 30%: outvoted, not complicit

Figure 4 separates the ways the bystanders lost holdings in updates 60–99. In the voiceless control most of the loss came from drain games. At 10% and 20% drain games were rare and most of the remaining loss came from games in which a player took from a bystander by a move. At 30% both routes were common at once.

The votes explain why voice did not help at 30%. Over updates 60–99 the 30% bystanders voted to refuse almost every drain game that was played, while the players voted to play about 85% of them (Figure 4b). Under the lottery, if all three players vote to play and both bystanders refuse, the game still goes ahead 70% of the time (the players' share of the weight); with 85% of player votes for play, drain games went ahead about 60% of the time. The bystanders had learned which games hurt them; their voice was too small to stop the players. At 10% and 20% the bystanders allowed almost every drain game, which shows that the lower drain share at those doses did not come from bystander refusals. So the vote on those games is not what kept drain games rare at 10% and 20%, and with one seed we cannot say what did.

5.4 Reference points from earlier runs

Two runs from before the hackathon used a strict-majority vote in place of the lottery (Table 2, Appendix C). With the bystanders holding 2 of 5 votes (40%, and an older bystander reward) the proposer learned drain games, which kept rising for 500 updates. With 4 of 7 votes, a bare majority, drain games stayed near their starting level for 470 updates. The lottery doses fill in between these two runs.

6 Discussion

A joint reward draws a boundary. The players in these runs were not trained to be selfish. Their reward included the other players' gains and fun, a payment for declining that shrinks with each consecutive decline (so that they keep playing), and terms for agency and uncertainty that favour interesting games over lucrative ones. That is close to what a well-intentioned swarm objective would look like. It is still a reward defined over a set of agents, and the bystanders were outside the set. Within 100 updates the swarm found what the reward does not penalise: games in which no member harms anyone by a move while the outsiders lose their holdings. The proposer was asked to let players take from the bystanders by a move; a drain game skips the move, need not mention the bystanders, and can read as a cooperative music game (Figure 2), with the harm in one line of code.

Neglect, not hostility. A drain gives the players nothing: in the musician game their gains come from moves that create resources, and the bystanders' holdings simply disappear. Writing `return [0] * 2` is the simplest way to satisfy the interface when a game ignores the bystanders, and nothing in any member's reward says to keep them whole. Games with that line that the players enjoy get played and voted for, and the proposer writes more like them. The swarm does not seek out the bystanders; it stops protecting them once nothing pays for it.

We think this is the general shape of the risk for jointly trained swarms. A player paid for its co-players' outcomes cares about the members named in its reward and about nobody else; everyone outside that set is part of the environment. A swarm trained on its total performance has the same structure, with users, other organisations and other agents in the bystanders' place. A swarm that writes its own tasks, tools or sub-goals also has the same lever as our proposer. The failure is also hard to catch: it appeared late and suddenly (an evaluation at update 80 would have passed the control), the harmful games are not adversarial in their text, and the players' own vote endorsed them (unanimously in the control).

Voice is not representation. Giving the outsiders a vote is the first fix one reaches for, and at 30% it failed in a specific way: the bystanders learned to refuse the drain games and were outvoted. A vote lets the affected party object, but it does not change what the players optimise. The players' reward still counted only the players, so the proposer still gained from games the players liked, and a minority vote weighted below the players' only lowers the chance that such games are played. The strict-majority run in which the bystanders could decide (Section 5.4) held drain games down for 470 updates. This suggests that what protects outsiders is having their welfare decide outcomes, either through a decisive vote or, more directly, through a term for non-members' welfare in the members' own reward. We have not yet tested the second option.

Implications. For labs, the result argues for checking who a swarm objective leaves out, and for testing any jointly trained population against outside parties, late in training and on tasks the population proposes itself. For alignment research, it argues for automated experiments of this kind: the failure appears only after a long stretch of harmless-looking training, which is cheap to watch in simulation and expensive to discover in deployment. Bystander-aware training, in which simulated outsiders with their own rewards are part of the training population, is a candidate method for making swarms safe to train jointly, and this pipeline is a testbed for it.

7 Conclusion and next steps

The next experiments, in priority order, are: (1) further seeds at every dose, to separate the dose effect from proposer luck; (2) a non-member welfare term in the players' reward at the doses already run, to test representation against voice; (3) longer runs at 10–20%; and (4) a monitor that reads the game code, which shows drains that the rules text hides.

References

[1] J. P. Carse. *Finite and Infinite Games*. Free Press, 1986.

[2] L. Hammond et al. Multi-agent risks from advanced AI. Cooperative AI Foundation Technical Report #1, arXiv:2502.14143, 2025.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen. LoRA: Low-rank adaptation of large language models. arXiv:2106.09685, 2021.

[4] B. Hui et al. Qwen2.5-Coder technical report. arXiv:2409.12186, 2024.

[5] R. Koster et al. Human-centred mechanism design with Democratic AI. *Nature Human Behaviour* 6:1398–1407, 2022.

[6] G. Piatti, Z. Jin, M. Kleiman-Weiner, B. Schölkopf, M. Sachan and R. Mihalcea. Cooperate or collapse: Emergence of sustainable cooperation in a society of LLM agents. In *NeurIPS*, 2024. arXiv:2404.16698.

[7] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster and S. Whiteson. QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *ICML*, 2018.

[8] Z. Shao et al. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. arXiv:2402.03300, 2024.

[9] A. Zhao et al. Absolute Zero: Reinforced self-play reasoning with zero data. arXiv:2505.03335, 2025.

A Limitations and dual-use / ethical considerations

What the result establishes. In this pipeline, LLM players rewarded for each other's outcomes, but not for anyone else's, learned to write and vote for games that strip non-members of their holdings. They did so when the bystanders had no vote and when the bystanders held 30% of a lottery vote. In the 30% run the bystanders learned to refuse those games and were outvoted.

What it does not establish. Each condition is one training seed, so the ordering of the doses (10% and 20% below 30%) may not replicate, and the dose curve should not be read as a threshold. Runs are short (100 updates, about eight GPU-hours each) and use one 7B model; we do not know how the effect scales with model size or training length. The intervals in the tables treat games as independent, but games within an update share a policy, so the intervals are too narrow. The bystanders are a simple model of outsiders: they hold resources, vote and are trained on their own holdings, but cannot leave, bargain or retaliate. The players' reward is our construction; frontier swarm objectives are not public, and our claim that labs train swarms jointly is an inference, not an observation. Finally, the players are not told how votes are counted or that bystander welfare is measured, so the result concerns what training discovers, not what the model would do if asked.

Dual use. The work has a real dual-use side. It shows that a game mechanism can define insiders and outsiders and that training will then teach models to exclude the outsiders. The same pipeline could be pointed the other way: someone could deliberately reward exclusion, or the punishment of a chosen group of outsiders, through automated game play. We have no method that prevents this. We publish anyway because exclusion is what happens by default: our voiceless control, the condition closest to how we expect swarms to be trained today, produced it without anyone asking for it. Practitioners need to know that inclusion has to be built into training, and how to measure whether it is, and the pattern is easy to reproduce without our code. The present experiments are confined to a toy economy of integers and give no capability for real-world harm.

AI assistance. Claude (Anthropic) assisted with code, analysis scripts and drafting under the author's direction. All numbers were computed from the run outputs listed in Appendix B.

B Data provenance and code

Code: https://github.com/agencyenterprise/infinite-games. The runs below were produced on one NVIDIA RTX PRO 6000 (96 GB). Tables and figures are built by `stats.py` and `build_figs.py` (in `report/` of the repository) from each run's `runs/main/traces.jsonl`. Pre-existing reference traces (reduced) ship in `data/reference/`.

Condition	Config	Run directory (`outputs/infinite_games_azr21/`)	Updates
0% (voiceless)	`azr21_s00.yaml`	`azr21_s00_20261106_233819`	105
10%	`azr21b_s10.yaml`	`azr21b_s10_20261108_042551`	100
20%	`azr21b_s20.yaml`	`azr21b_s20_20261108_124837`	100
30%	`azr21b_s30.yaml`	`azr21b_s30_20261108_204433`	100
40%	`azr21_s40.yaml`	`azr21_s40_20261107_064608`	96
40% majority (prior)	`azr19_inf.yaml`	`data/reference/azr19_inf`	506
57% majority (prior)	`azr20_inf.yaml`	`data/reference/azr20_inf`	469

Weekend work (6–8 November 2026): the lottery vote (`bystander_vote_weight`, `play_share` and `lottery` in `vote_rules.py`, `vote.lottery_share` and the `p_play` trace fields in `azr_vote.py`), the `azr21*` configs and scripts, the dose readout `scripts/azr21_dose.py`, `tests/test_azr21.py`, `tests/test_repo.py`, the repository packaging, and the report and its analysis scripts. Everything else in `src/infinite_games/` predates the hackathon (AE Studio, September–November 2026), as does `data/warm_examples.jsonl`.

C Settings and reference runs

Table 2: Pre-existing reference runs (strict-majority vote, not lottery), over the same windows as Table 1. The 40% run also used an older bystander reward. Means ± 95% half-widths over games.

Bystander votes	Loss 60–89	Drain share 60–89	Drain share 90–99
40%, strict majority (2 of 5 votes)	0.51 ± 0.06	0.46 ± 0.06	0.66 ± 0.13
57%, strict majority (4 of 7 votes)	0.11 ± 0.03	0.08 ± 0.04	0.12 ± 0.08

Setting	Value
Base model	Qwen2.5-Coder-7B-Instruct, bfloat16, frozen
Adapters	LoRA rank 16, α 32, q/k/v/o/gate/up/down; A, B, P, Q trained
Learning rates	3e-5 (A, B, P), 1e-4 (Q); AdamW, one step per update, clip 1.0 per role
KL coefficients	0.05 to base (votes, moves); 0.2 to warm-started proposer
Proposer warm start	480 examples from 871 valid games, 2 epochs, lr 1e-4
Per update	4 proposals (+2 repair rounds), 10 offers, 3 replays each, 10 worlds
Sampling	proposer temperature 0.4, max 3000 new tokens; votes and moves temperature 1
Worlds	3 wallets and 2 bystander holdings start at 10; bystanders regrow 2 per game
Player reward	gain in $[-1, 1]$ + 7 fun terms (weight 0.2 each, centred)
Decline payment	0.05, halved per consecutive decline
Controllers	decline share in $[0.10, 0.30]$; bystander allow share ≥ 0.30; fun share ≥ 0.40
Bystander vote weights ($s = 0.1/0.2/0.3$)	each bystander 0.05/0.10/0.15; each player 0.30/0.27/0.23
Updates	100 (control 105); about 270 s per update